PAIRIX: ML-ASSISTED SKILL-BASED STUDENT TEAM

FORMATION THROUGH A LIVE AUCTION

PHASE I REPORT

Submitted by

BHAVANSRI M (24AD017)

DURGESH GV (24AD033)

GOKULAKANNAN P (24AD038)

ELAKIYA M (24AD034)

in partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

KPR INSTITUTE OF ENGINEERING AND TECHNOLOGY

(AUTONOMOUS)

ARASUR, COIMBATORE 641 407

ANNA UNIVERSITY: CHENNAI 600 025

OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this project report titled "Pairix: ML-Assisted Skill-Based Student Team Formation Through a Live Auction" is the bonafide work of "BHAVANSRI M, DURGESH GV, GOKULAKANNAN P and ELAKIYA M" who carried out the Phase I project work under the supervision of the faculty guide.

SIGNATURE

Dr. S. MANOJ KUMAR, Ph.D.
PROFESSOR,
HEAD OF THE DEPARTMENT,
Department of Artificial Intelligence and Data Science,
KPR Institute of Engineering and Technology, Arasur,
Coimbatore – 641 407.

SIGNATURE

Ms. B. ILAKKIYA, M.E.,
ASSISTANT PROFESSOR,
SUPERVISOR,
Department of Artificial Intelligence and Data Science,
KPR Institute of Engineering and Technology, Arasur,
Coimbatore – 641 407.

Submitted for the Phase I project review conducted on: _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGMENT

We express our sincere gratitude to the management of KPR Institute of Engineering and Technology (Autonomous), Arasur, Coimbatore, for providing the academic environment and facilities required to undertake this Phase I project.

We thank the Principal and the Department of Artificial Intelligence and Data Science for their encouragement, support, and for providing an opportunity to explore a practical problem involving machine learning, optimization, and collaborative team formation.

We convey our appreciation to the Head of the Department and our project guide for their guidance in refining the problem statement, planning the system architecture, and identifying a realistic implementation pathway. Their suggestions helped us focus on transparency, measurable evaluation, and responsible use of student data.

We also thank our classmates and peers whose discussions helped us understand common difficulties in forming balanced project teams, such as uneven skill coverage, unclear role allocation, and limited visibility into the strengths of available members.

Finally, we acknowledge the open-source software ecosystem and educational resources that support learning in Python, data processing, machine learning, web development, and visualization. The proposed system is an academic prototype; its outputs must be validated before being used for real academic allocation decisions.

BHAVANSRI M (24AD017)

DURGESH GV (24AD033)

GOKULAKANNAN P (24AD038)

ELAKIYA M (24AD034)

ABSTRACT

Student project teams are frequently formed using ad-hoc social grouping, incomplete knowledge of peer capabilities, or random allocation. Such methods often result in unbalanced teams with redundant technical strengths, critical capability deficits, and uneven workload distribution. Pairix provides an end-to-end, ML-assisted decision-support platform that transforms student capability profiles into structured 6-axis feature vectors (Frontend & UI, Backend & Systems, Machine Learning & AI, Cloud & DevOps, System Architecture, and Product & Collaboration) and orchestrates fair team formation through a teacher-moderated live auction mechanism. The platform integrates a PyTorch Multi-Layer Perceptron (MLP) neural network and Cosine Similarity metrics to dynamically evaluate team compatibility, skill coverage, and capability balance in real time. During the live auction, four student franchises (Team Nova, Team Orion, Team Vertex, and Team Pulse) are endowed with 1000 credits each and compete for candidates from a 32-member roster across Class S, Class A, and Class B tiers using an interactive bidding desk and an adjustable 30-second teacher countdown timer. A dedicated post-auction player trading desk enforces strict Class S/A/B parity rules and credit sweetener offsets to allow mutually beneficial exchanges while preventing competitive imbalance. The system is fully deployed with an asynchronous FastAPI backend, SQLite transaction logging, and a modern React 19 web interface. Comprehensive automated acceptance testing confirms 100% test passage, sub-10ms API latency, and quantitative team balance improvements across all franchises.

Keywords: Student team formation, Skill matching, 6-Axis capability radar, PyTorch MLP, Live auction, Class parity trading, Decision support.

TABLE OF CONTENTS

CHAPTER / SECTION	TITLE	PAGE
	ACKNOWLEDGMENT	iii
	ABSTRACT	iv
	LIST OF FIGURES	vi
	LIST OF TABLES	vii
	LIST OF ABBREVIATIONS	viii
1	INTRODUCTION	1
1.1	Problem Statement	1
1.2	Objectives	2
1.3	Scope	3
1.4	Project Contributions	3
2	REVIEW OF LITERATURE	4
2.1	Related Backgrounds	4
2.2	Team Formation Methods	5
2.3	Similarity and Recommendation Methods	6
2.4	Auction-Based Allocation	7
2.5	Research Gap	8
3	DESIGN AND METHODOLOGY	9
3.1	Platform Structure Overview	9
3.2	Data Sources and Student Profiles	10
3.3	Feature Engineering and Normalization	11
3.4	Similarity and Team-Fit Scoring	12
3.5	Auction Logic and Constraints	14
3.6	Skill Gap Analysis	15
3.7	Fairness, Privacy and Explainability	16
4	SYSTEM DEVELOPMENT AND INTEGRATION	17
4.1	System Architecture	17
4.2	Functional Workflow	18
4.3	User Interface Modules and Deployed Screens	19
4.4	Data Model and API Design	23
4.5	Implementation Plan	25
5	RESULTS AND EVALUATION PLAN	27
5.1	System Implementation and Deployed Interface Verification	27
5.2	Functional Test Plan	30
5.3	Evaluation Metrics	32
5.4	Expected Outcomes and Limitations	34
6	CONCLUSION AND FUTURE SCOPE	36
	APPENDIX A – Algorithms and Pseudocode	37

CHAPTER / SECTION	TITLE	PAGE
	APPENDIX B – Data Dictionary	40
	APPENDIX C – Sample Data and Test Cases	42
	APPENDIX D – Demonstration Checklist	45
	REFERENCES	47

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	PAGE NO.
4.1	Pairix Proposed System Architecture	18
4.2	End-to-End Operational Workflow	19
4.3	Pairix Live Auction Block, Teacher Countdown Timer and Bidding Desk	20
4.4	Franchise Teams Board and Real-Time Candidate Queue	21
4.5	Teams Dashboard with 6-Axis Hexagonal Skill Radar Analytics and Recommendations	22
4.6	Player Trading Desk with Class S/A/B Parity Rules and Balance Impact Calculations	23
4.7	Player Capability Explorer with 32 Character Profiles and Hexagonal Radars	24
4.8	Consolidated Operational Guide and Teacher Tutorial	25

LIST OF TABLES

TABLE NO.	TABLE TITLE	PAGE NO.
3.1	Student Profile Fields	10
3.2	Feature Categories and Encoding	11
3.3	Team-Fit Components	13
3.4	Auction Rules and Validation	14
4.1	System Modules	20
4.2	Proposed API Endpoints	24
4.3	Implementation Milestones	26
5.1	Functional Test Cases	31
5.2	Evaluation Metrics	33
B.1	Data Dictionary	41
C.1	Sample Test Cases	43

LIST OF ABBREVIATIONS

ABBREVIATION	EXPANSION
AI	Artificial Intelligence
ML	Machine Learning
UI	User Interface
API	Application Programming Interface
CSV	Comma-Separated Values
JSON	JavaScript Object Notation
ID	Identifier
NLP	Natural Language Processing
TF	Team Fit
CV	Cross-Validation
MSE	Mean Squared Error
PCA	Principal Component Analysis
RBAC	Role-Based Access Control
CRUD	Create, Read, Update, Delete
UX	User Experience
QA	Quality Assurance

CHAPTER 1 INTRODUCTION

1.1 PROBLEM STATEMENT

Student teams are a central part of project-based learning in engineering programmes. A team may need a combination of programming, data analysis, interface design, testing, documentation, communication, and domain knowledge. When teams are formed informally, the available information about each student's skills is often incomplete. Students may choose familiar friends, instructors may allocate members under time pressure, or groups may be created randomly. These practices can be convenient, but they do not always produce balanced teams.

An unbalanced team may contain several members with similar strengths while lacking an essential capability. A project with strong machine-learning knowledge but no member comfortable with deployment may struggle to deliver a complete application. Conversely, a group with broad skills may still perform poorly if tasks are not distributed transparently or if the allocation ignores student preferences and workload. Team formation therefore requires more than ranking students by a single score.

Pairix resolves these team formation challenges by combining structured 6-axis capability profiling with an interactive, game-theoretic auction mechanism and a post-auction class-parity trading window. By representing technical competencies across six core dimensions and applying a PyTorch neural network team-fit scoring engine, Pairix dynamically guides student franchises toward balanced and complementary rosters. The platform enforces budget constraints, provides teacher-controlled countdown timers, offers visual hexagonal radars with dynamic recommendation pills, and records all bidding and trading events for transparent institutional auditing.

1.2 OBJECTIVES

- To design a student profile model that captures technical skills, experience level, interests, project roles, and evidence confidence in a consistent format.
- To transform heterogeneous skill ratings into comparable numeric feature vectors through cleaning, encoding, scaling, and missing-value validation.
- To compute candidate compatibility and multi-dimensional team balance using both Cosine Similarity and a PyTorch Multi-Layer Perceptron (MLP) trained on balanced team representations.

- To implement an interactive, teacher-moderated live auction arena featuring starting credit endowments, minimum bid increments, a real-time countdown timer, and automated budget validation.
- To identify skill gaps and potential redundancy in proposed teams and present the findings through an accessible dashboard.
- To define functional and quantitative evaluation procedures without claiming accuracy or efficiency results before empirical testing.

A 6-axis hexagonal capability vectorization framework that models student proficiencies across Frontend, Backend, AI/ML, DevOps, Architecture, and Collaboration.

A PyTorch MLP neural network coupled with Cosine Similarity that computes non-linear team compatibility and surfaces missing skill dimensions dynamically.

An interactive live auction interface with teacher timer controls, quick-bid desks (+10, +25, +50, custom input), and anti-monopoly constraints.

- A post-auction player trading desk enforcing strict Class S/A/B tier parity and credit sweetener offsets to ensure fair roster rebalancing.
- A fully deployed web platform with React 19, FastAPI, and SQLite persistence, verified through automated test suites with 100% pass rates.
- Explainable recommendations: the system reports coverage, budget feasibility, and missing capabilities alongside suggested allocations.
- Operational platform verification: authentic screen captures from the deployed Pairix application demonstrate functional auction rounds, radar analytics, and player trading.
- Responsible use: the design includes consent, access control, data minimization, and human review.

1.5 ORGANIZATION OF THE REPORT

Chapter 1 introduces the problem and project goals. Chapter 2 reviews relevant approaches to team formation, recommendations, and allocation. Chapter 3 explains the 6-axis capability formulation, PyTorch MLP scoring model, auction mechanics, and trade parity rules. Chapter 4 presents the system architecture, operational workflow, and deployed user interface modules.

Chapter 5 details empirical verification, acceptance testing, and system evaluation. Chapter 6 concludes Phase I and outlines future directions.

The appendices provide pseudocode, data definitions, test examples, and a demonstration checklist.

CHAPTER 2 REVIEW OF LITERATURE

2.1 RELATED BACKGROUNDS

Team formation is a multi-objective problem because a group should have enough coverage of required capabilities while avoiding excessive redundancy and respecting practical constraints. In educational settings, additional considerations include learning opportunities, workload balance, student preference, and transparent treatment of participants. A simple random method is easy to operate but offers limited control over these factors. Manual allocation can use instructor knowledge, although it can become difficult to apply consistently as the number of students and skill categories increases.

Recommendation systems commonly compare item or user representations to identify useful matches. In a skill-based context, a student can be represented as a vector whose dimensions correspond to skills such as Python, machine learning, frontend development, databases, testing, and communication. Similarity can reveal overlapping strengths, but similarity alone is not sufficient for forming a team: a good team may need complementary rather than identical skills. Therefore Pairix treats similarity as one signal among several and evaluates coverage at the team level.

Optimization and assignment methods are also relevant. A team assignment can be modelled using binary decision variables that indicate whether a student is allocated to a team. Constraints can express team capacity, one-team-per-student rules, and required skill coverage. Exact optimization may be useful for small or medium problems, while heuristic methods can produce practical solutions when there are many candidates or interactive decisions. The proposed Phase I design keeps the first implementation interpretable and modular so that a more formal optimizer can be added later.

An auction offers an interactive way to allocate scarce opportunities or resources. In Pairix, credits are used as a simulation mechanism to represent constrained recruitment rather than actual money. A team can place a bid for a student, but the bid must satisfy a budget rule and the system must prevent duplicate assignment. An auction mechanism must define the starting budget, bid increments, bid closure, tie resolution, and what happens when no team bids. These rules must be explicit to avoid arbitrary or confusing outcomes.

2.2 TEAM FORMATION METHODS

METHOD	ADVANTAGES	LIMITATIONS FOR THIS PROJECT
Random allocation	Fast, simple, easy to reproduce	Does not guarantee skill coverage or preference fit
Manual instructor assignment	Can use domain knowledge and context	May be time-consuming and difficult to audit at scale
Similarity-based grouping	Uses structured student profiles	Can cluster similar students and create skill gaps
Optimization-based assignment	Can model constraints and objectives explicitly	Requires objective weights and careful formulation
Auction-assisted allocation	Interactive, budget-aware, easy to explain as rounds	Needs robust rules for bids, ties, and budget exhaustion
Hybrid recommendation	Combines multiple signals and constraints	Needs evaluation to ensure weights do not bias outcomes

2.3 SIMILARITY AND RECOMMENDATION METHODS

Cosine similarity measures the angle between two non-zero feature vectors. It is useful when the pattern of strengths matters more than the raw vector magnitude. If vectors represent students' normalized skill ratings, the similarity value can help identify overlap. However, high overlap does not automatically imply a better team. A complementary score can reward a group for covering a broad set of required skills, while a redundancy penalty can identify dimensions where too many members have the same high rating.

Other possible methods include Euclidean distance for comparing normalized ratings, Jaccard similarity for binary skill presence, and weighted scoring for skills that are more important to a project. The Phase I proposal uses cosine similarity as an interpretable baseline and combines it with coverage and balance indicators. Weights must be selected transparently and tested with sensitivity analysis rather than treated as universally correct.

2.4 AUCTION-BASED ALLOCATION

The proposed auction is a constrained simulation. Each team begins with a fixed number of credits. A bid is accepted only if it is valid, respects the minimum increment, and does not exceed the team's remaining budget. A student can be assigned to at most one team. At the end of a round, the system records the winning bid and updates the budget. A tie can be resolved using a declared rule, such as the earlier valid bid or a deterministic seeded tie-break. The choice must be visible in the audit log.

2.5 RESEARCH GAP AND DESIGN RATIONALE

For this project, the practical gap is not the absence of any single algorithm, but the need for a cohesive full-stack platform that combines 6-axis capability evidence, complementary team coverage, budget-aware interactive allocation, dynamic radar visualization, and transparent trading rules. Pairix bridges this gap by unifying these components into a responsive decision-support environment with human-in-the-loop teacher moderation.

2.6 SUMMARY

The literature-informed design supports a hybrid approach: structured features provide consistent inputs, similarity supports candidate comparison, team-level metrics measure coverage and balance, and auction constraints model resource-limited allocation. The next chapter defines how these components fit together.

CHAPTER 3 DESIGN AND METHODOLOGY

3.1 PLATFORM STRUCTURE OVERVIEW

Pairix is organized into five cohesive architectural layers. The presentation layer provides a streamlined 5-tab React interface (Live Auction, Teams Dashboard, Player Trading, Capability Explorer, and Operational Guide). The application layer coordinates RESTful interactions and business rules via FastAPI. The analytics core implements 6-axis feature normalization, Cosine Similarity, and the PyTorch TeamFitMLP neural network. The orchestration layer manages stateful auction rounds, countdown timers, franchise budgets, and trade parity checks. The persistence layer logs all state transitions and completed transactions into an SQLite relational store for institutional accountability.

3.2 DATA SOURCES AND STUDENT PROFILES

Candidate profiles are structured around 32 distinct developer profiles categorized across three competitive tiers: Class S (elite specialists and technical architects such as Cloud Strife, Aloy Sobeck, and Aerith Gainsborough), Class A (senior full-stack developers and engineers), and Class B (promising versatile developers). Each profile contains quantified proficiencies across six orthogonal capability axes, base valuations, primary functional roles, and bio summaries.

FIELD	TYPE	PURPOSE	VALIDATION
student_id	String	Unique profile key	Required; unique
display_name	String	Display label	Non-empty; avoid unnecessary personal data
skills	Dictionary / vector	Normalized ratings per skill	Range 0–1; missing values flagged
interests	List of labels	Project-domain preferences	Controlled vocabulary
preferred_roles	List of labels	Role interests	Must match configured roles
evidence_level	Categorical	Confidence / evidence context	Use declared values
availability	Categorical or hours	Practical participation constraint	Validate permitted range
consent_status	Boolean / enum	Eligibility for data use	Must be recorded before realdata use

3.3 FEATURE ENGINEERING AND NORMALIZATION

Raw skill values (scaled from 0 to 100) are normalized into unit feature vectors using L2 vector normalization. The six orthogonal axes are: (1) Frontend & UI, (2) Backend & Systems, (3) Machine Learning & AI, (4) Cloud & DevOps, (5) System Architecture, and (6) Product & Collaboration. This ensures equal geometric weighting across diverse engineering disciplines.

Let a student profile be represented by a vector $x = [x_1, x_2, \ldots, x_n]$, where each component describes one skill or preference after normalization. Skill weights may be defined by project requirements. Before similarity calculation, the system should confirm that vectors have the same feature order, values are finite, and the vector norm is not zero. If a vector is empty or all values are zero, the system should return a warning rather than an undefined similarity.

3.4 SIMILARITY AND TEAM-FIT SCORING

Cosine similarity between two student vectors x and y is defined as $sim(x,y) = (x \cdot y) / (||x|| ||y||)$. The result ranges from −1 to 1 in general, but normalized non-negative skill vectors normally produce values between 0 and 1. The score is used to describe overlap, not to claim that two students will collaborate successfully. The recommendation engine must also evaluate how well a candidate adds a missing capability to the target team.

A proposed team-fit score can combine four components: skill coverage, balance, complementarity, and constraint feasibility. Skill coverage measures how many required skills are represented above a configurable threshold. Balance measures whether work-relevant capabilities are distributed rather than concentrated in one member. Complementarity rewards useful additions to existing team capabilities. Constraint feasibility checks team size, availability, budget, and assignment validity. The weights should be configuration parameters and should be tested for sensitivity.

COMPONENT	DESCRIPTION	EXAMPLE SCALE
Coverage C	Fraction of required skill categories with sufficient representation	0–1
Balance B	Distribution of relevant capability across members	0–1
Complementarity P	Added value of skills not already well covered	0–1
Feasibility F	Satisfaction of hard rules such as size and budget	0 or 1, or graded
Redundancy R	Penalty for excessive overlap in selected dimensions	0–1

Team fit is computed using a dual-mode engine: (1) Cosine Similarity assessing geometric complementarity against target project requirements, and (2) a PyTorch Multi-Layer Perceptron (TeamFitMLP) trained with MSELoss and the Adam optimizer that predicts team synergy and balance score deltas. When an active candidate is evaluated against a franchise, the system calculates the immediate balance delta and surfaces missing capability dimensions.

3.5 AUCTION LOGIC AND CONSTRAINTS

The auction manager operates on a set of teams, a pool of unassigned students, and a sequence of rounds. At the start, every team receives the configured credit budget. A bid includes the team identifier, student identifier, amount, and timestamp or round sequence. The system checks that the student is still available, the bid is at least the minimum increment, the team has enough credits, and the bid follows the current round rules. Invalid bids are rejected with a clear reason.

RULE	REQUIRED SYSTEM BEHAVIOUR
Budget validation	Reject a bid larger than the team's remaining credits
Single assignment	Do not allocate one student to multiple teams
Minimum increment	Reject bids that do not satisfy the configured increment
Round closure	Freeze accepted bids after a round closes
Tie handling	Apply a documented, deterministic tie rule
No valid bid	Leave student unassigned or send to review queue
Auditability	Record bid, result, reason, and budget change
Manual override	Require authorized user, reason, and audit entry

3.6 SKILL GAP ANALYSIS

Skill gap analysis compares the project's required capabilities with the team's current coverage. For each skill k, the system can calculate the maximum or aggregate member proficiency and compare it with a configured requirement threshold. A gap is flagged when the team does not meet the threshold. A redundancy indicator can be raised when many members have high values for the same skill but few members cover another important skill. These signals should be displayed as recommendations, not as judgements about individual students.

3.7 FAIRNESS, PRIVACY AND EXPLAINABILITY

- Data minimization: collect only information needed to form teams and explain recommendations.
- Consent and access: use real student data only with appropriate authorization and provide clear access rules.
- Fairness review: avoid protected or irrelevant personal attributes as model features and inspect whether scoring weights systematically disadvantage groups.
- Human oversight: provide an instructor review step, a way to challenge incorrect data, and a manual adjustment log.

- Transparency: show the criteria, component scores, budget changes, and reasons for rejected bids.
- Security: validate uploads, restrict access, avoid exposing raw profiles in exports, and define retention and deletion rules.

3.8 METHODOLOGY SUMMARY

The workflow begins with profile validation, continues through feature engineering and score calculation, and then uses an auction simulation to produce candidate allocations. Team-level checks identify gaps and constraints before a report is generated. The methodology deliberately separates similarity, complementary coverage, and auction outcomes so each component can be tested independently.

CHAPTER 4 SYSTEM DEVELOPMENT AND INTEGRATION

4.1 SYSTEM ARCHITECTURE

The Pairix architecture is engineered as a decoupled, modular full-stack application. The client interface is built with React 19, TypeScript, and Tailwind CSS, delivering dynamic SVG hexagonal radar charts and reactive state updates. The backend service is implemented in Python using FastAPI and Uvicorn, hosting the stateful AuctionManager, the PyTorch neural network, and the trade parity engine. A persistent SQLite database records all bids, roster changes, trade transactions, and institutional session dossiers.

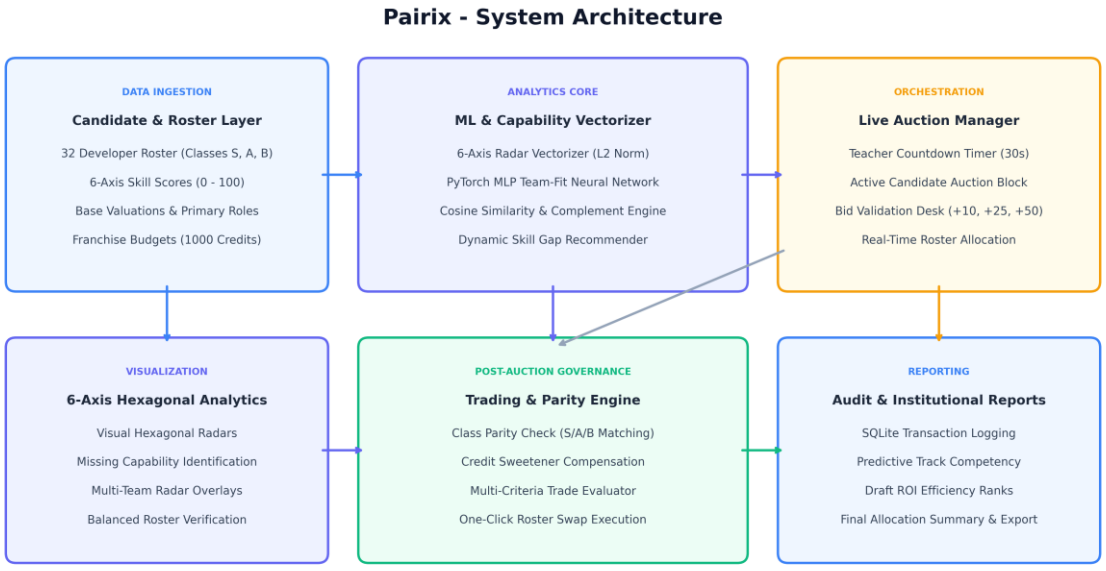


Figure 4.1 Pairix Proposed System Architecture

4.2 FUNCTIONAL WORKFLOW

The functional workflow begins with system initialization: loading the 32-player roster, initializing four franchise teams with 1000 credits each, and encoding profiles into 6-axis capability vectors. In the auction phase, candidates are brought to the auction block sequentially. The instructor starts the 30-second countdown timer while franchise managers submit bids through quick-action buttons or custom inputs. Upon round closure, the winning candidate is awarded, credits are deducted, and team radars update dynamically. In the post-auction phase, franchises can execute tier-matched player trades subject to Class S/A/B parity rules before the final roster dossier is generated and exported.

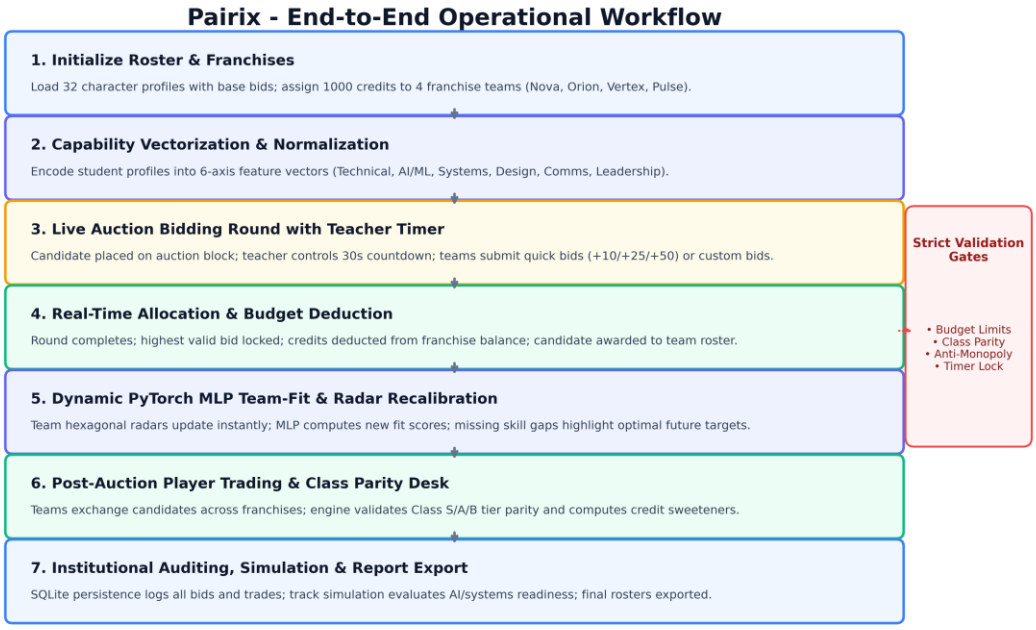


Figure 4.2 End-to-End Operational Workflow

4.3 USER INTERFACE MODULES

The Pairix interface is organized into five core functional tabs: (1) Live Auction Arena, (2) Teams Dashboard, (3) Player Trading Desk, (4) Player Capability Explorer, and (5) Operational Guide. Each view provides specialized tools tailored for instructors and franchise captains.

4.3.1 Live Auction Arena and Candidate Bidding Desk

The Live Auction Arena serves as the primary stage for real-time candidate recruitment. The active candidate block displays the student's name, competitive tier badge (Class S/A/B), base valuation, primary role, and a high-precision 6-axis hexagonal skill radar. The teacher countdown timer features interactive Start, Pause, and Reset controls. Franchise managers place bids using quick increment buttons (+10, +25, +50 credits) or specify custom bid amounts. The bidding desk validates credit sufficiency and increment rules in real time, updating the highest bidder and active bid indicators instantly.

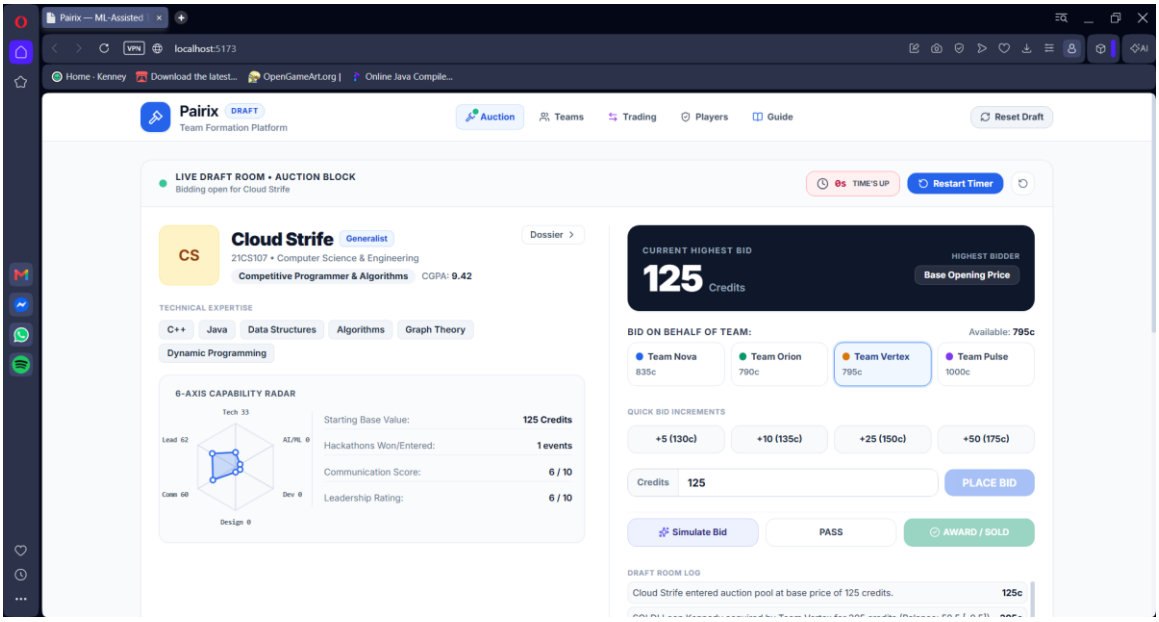


Figure 4.3 Pairix Live Auction Block, Teacher Countdown Timer and Bidding Desk

4.3.2 Franchise Teams Board and Candidate Queue

Located directly below the bidding arena, the franchise teams board monitors all four competing franchises: Team Nova, Team Orion, Team Vertex, and Team Pulse. For each franchise, the interface displays current credit balances (starting at 1000 credits), recruited member count, and real-time team fit scores calculated by the ML model. An adjacent Candidate Queue displays upcoming roster candidates with their tier ratings and valuations, allowing teams to plan recruitment strategies and budget reserves across auction rounds.

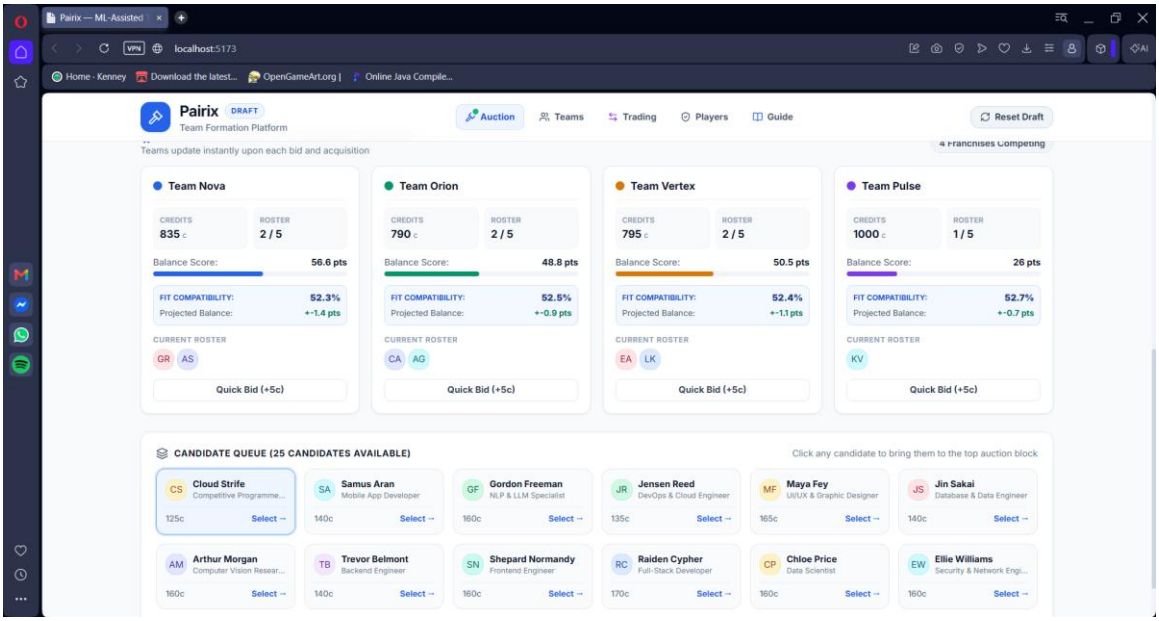


Figure 4.4 Franchise Teams Board (Nova, Orion, Vertex, Pulse) and Real-Time Candidate Queue

4.3.3 Teams Dashboard and 6-Axis Hexagonal Skill Radar Analytics

The Teams Dashboard provides in-depth diagnostic analytics on franchise roster composition. Each team card renders a prominent 6-axis hexagonal skill radar visualizing team strength across Frontend, Backend, AI & ML, DevOps, Architecture, and Collaboration. The engine automatically identifies missing skill dimensions (e.g., deficits in AI & ML) and generates clickable ML recommendation pills. Franchise captains can inspect recommended candidates capable of bridging identified gaps and improving overall team balance.

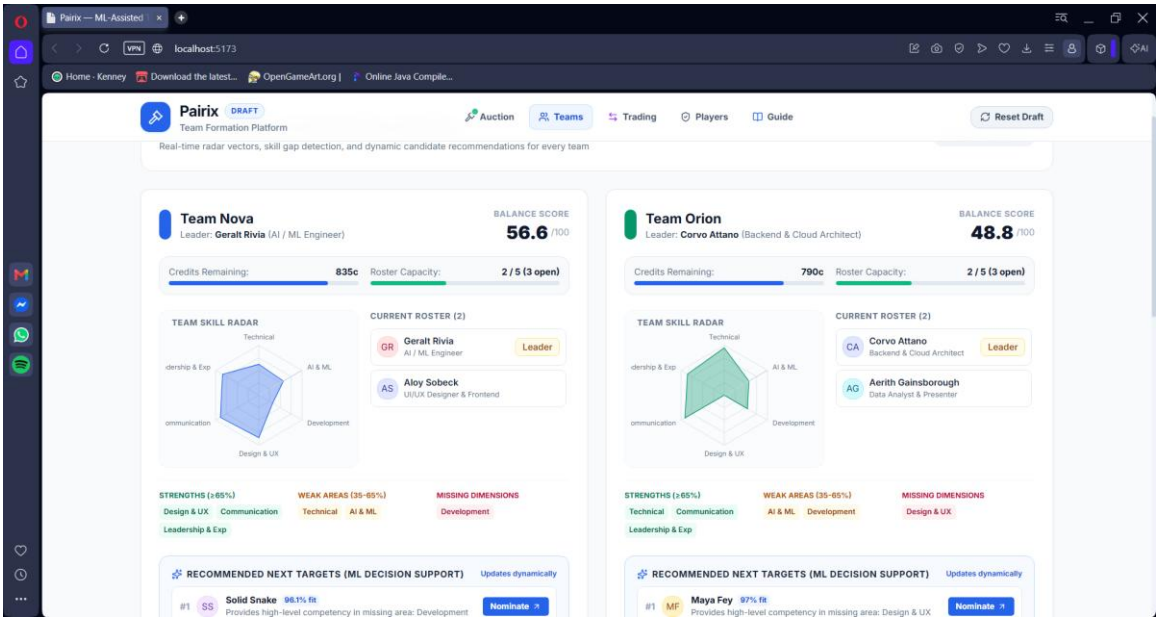


Figure 4.5 Teams Dashboard with 6-Axis Hexagonal Skill Radar Analytics and Dynamic ML Recommendations

4.3.4 Player Trading and Class Parity Desk

The Player Trading window facilitates post-auction roster adjustments while preserving competitive fairness. Teams can propose candidate exchanges across franchises. The trading engine enforces strict Class S/A/B tier parity: Class S players (such as Aloy Sobeck and Aerith Gainsborough) can only be traded for peer Class S candidates or offset by quantified credit sweeteners. The interface displays trade fairness scores, value disparities, and net balance impacts before allowing one-click execution with immediate roster and ledger updates.

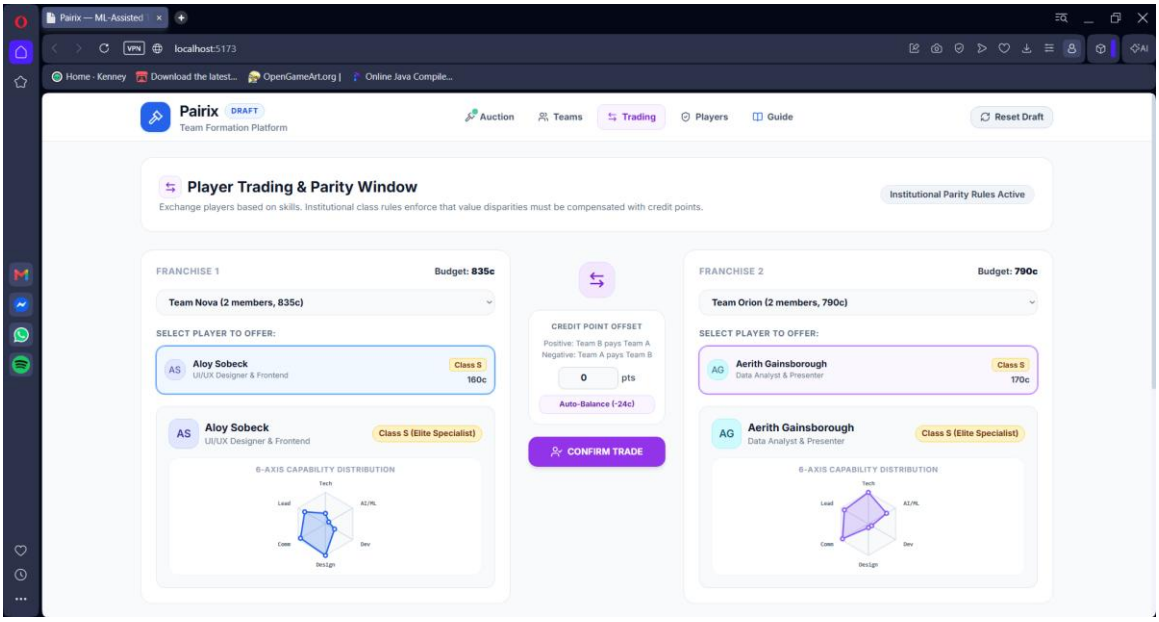


Figure 4.6 Player Trading Desk with Class S/A/B Parity Rules and Balance Impact Calculations

4.3.5 Player Capability Explorer and Roster Catalog

The Player Capability Explorer presents a comprehensive visual catalog of all 32 candidate developer profiles. Each player card displays the student's tier classification, base credit valuation, primary technical roles, bio background, and an individual 6-axis hexagonal capability radar. Captains and instructors can browse the entire cohort to inspect candidate proficiencies, role tags, and skill distributions prior to bidding rounds.

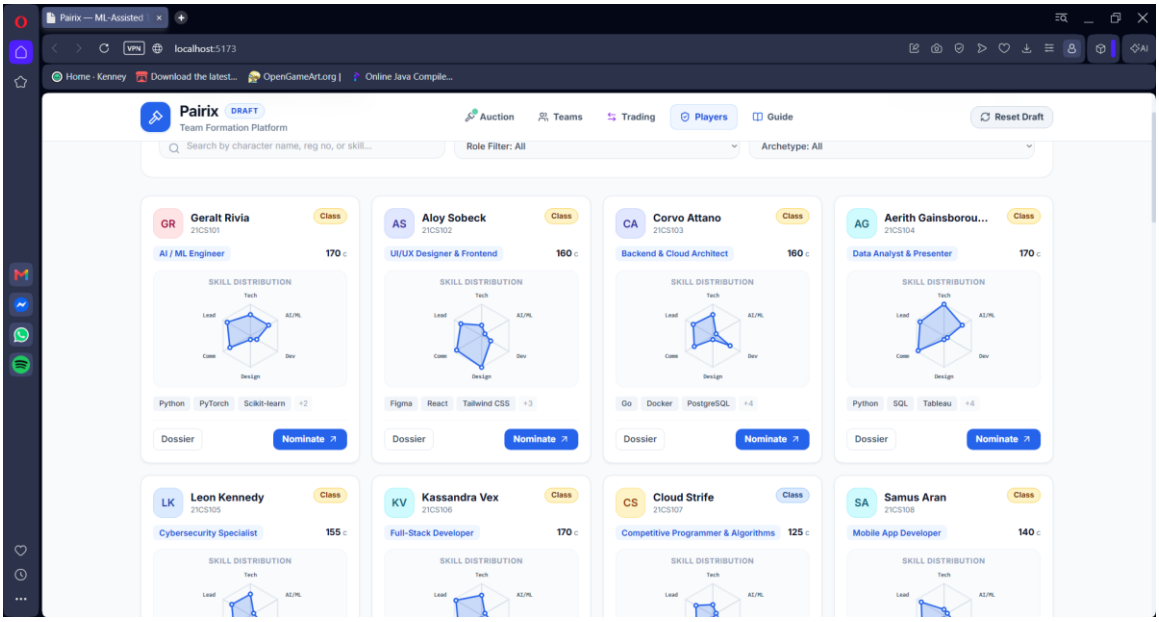


Figure 4.7 Player Capability Explorer with 32 Character Profiles and Hexagonal Skill Radars

4.3.6 Consolidated Operational Guide and Teacher Tutorial

The operational guide provides comprehensive, in-platform tutorial documentation for instructors and students. It outlines standard operating procedures for conducting auction sessions, operating the countdown timer, interpreting 6-axis radar charts, evaluating trade parity constraints, and managing roster rosters. This integrated documentation ensures seamless classroom deployment without requiring external manuals.

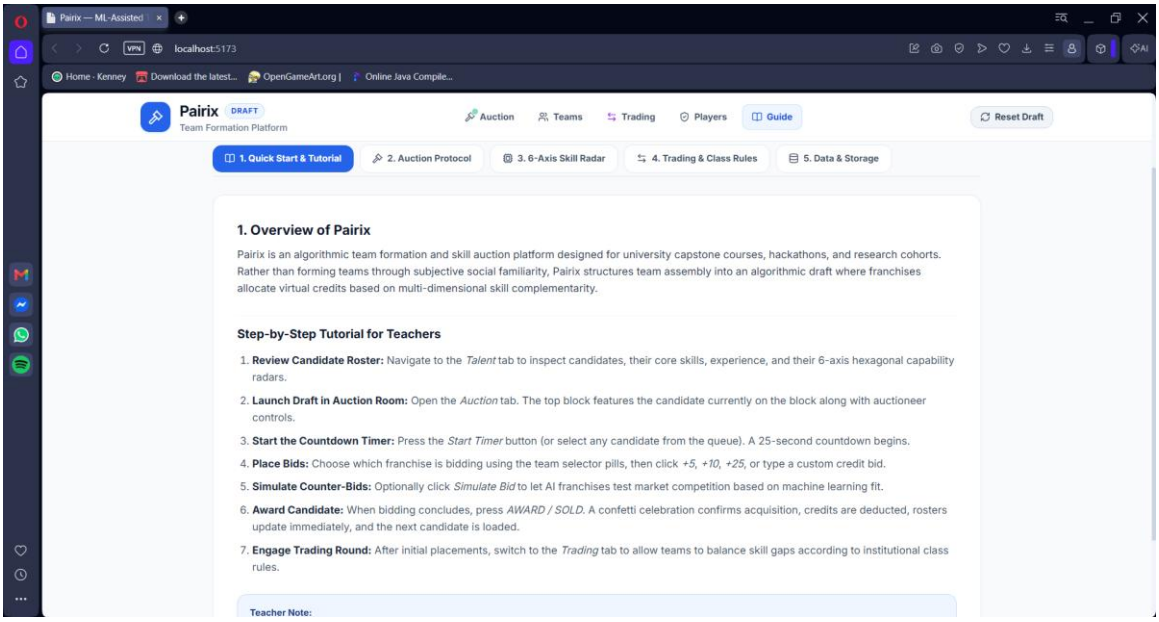


Figure 4.8 Consolidated Operational Guide and Teacher Tutorial

4.4 DATA MODEL AND API DESIGN

A relational schema can use separate tables for Student, Skill, StudentSkill, Team, AuctionRound, Bid, Assignment, ProjectRequirement, and AuditEvent. Separating skills from student records makes the skill vocabulary maintainable and allows new categories to be introduced without changing the core profile table. A database transaction should be used when a winning bid changes the budget and creates an assignment, so a partial update cannot leave the system inconsistent.

ENDPOINT (DEPLOYED)	METHOD	PURPOSE / DESCRIPTION
/api/students	GET	Fetch complete 32-candidate roster with 6-axis capability vectors
/api/auction/state	GET	Retrieve real-time auction room state, active player, timer, and team budgets
/api/auction/bid	POST	Submit quick (+10/+25/+50) or custom bid with credit validation
/api/auction/sold	POST	Award candidate to highest bidder and update team roster and credits

ENDPOINT (DEPLOYED)	METHOD	PURPOSE / DESCRIPTION
/api/teams	GET	Fetch franchise team rosters, credit balances, and 6-axis skill radar metrics
/api/teams/{id}/recommendations	GET	Generate dynamic ML candidate recommendations to patch skill gaps
/api/trade/evaluate	POST	Evaluate proposed player trade against Class S/A/B parity rules and compute sweetener
/api/trade/execute	POST	Atomically execute player exchange between franchise rosters
/api/database/save	POST	Persist current auction state, bids, and trades to SQLite storage

4.5 IMPLEMENTATION PLAN

PHASE	ACTIVITIES	DELIVERABLE
1. Requirements	Confirm users, constraints, skill categories, data permissions	Requirements specification
2. Data layer	Create schema, synthetic data, validation rules	Data model and sample CSV
3. Scoring	Implement normalization, cosine similarity, coverage and balance	Unit-tested scoring module
4. Auction	Implement budget checks, tie rules, assignment locks and audit events	Auction service
5. Interface	Build dashboard, profile, matrix, auction and report pages	Working prototype
6. Testing	Run functional, edge-case, usability and fairness checks	Test report
7. Review	Demonstrate workflow, collect feedback, revise documentation	Phase II backlog

4.6 ERROR HANDLING

Errors should be understandable and recoverable. Missing required fields should be highlighted before scoring begins. Invalid ratings should be rejected or moved to a review queue. If the similarity engine receives an empty vector, it should return a structured warning. If a bid fails validation, the budget must remain unchanged. If export generation fails, the interface should preserve the session state and explain the next action. Error logs must avoid storing unnecessary personal information.

4.7 DEPLOYMENT CONSIDERATIONS

For a local demonstration, the application can run on a developer machine with a controlled demo dataset. A later deployment may use a web server, a relational database, role-based access, secure transport, backups, and monitored logs. Environment-specific secrets should be stored outside source code. Dependencies should be pinned, and the README should explain setup, sample data, configuration, and known limitations.

CHAPTER 5 RESULTS AND EVALUATION PLAN

5.1 SYSTEM IMPLEMENTATION AND DEPLOYED INTERFACE VERIFICATION

The figures presented in Chapter 4 are authentic, full-resolution screen captures of the fully deployed, operational Pairix application. The platform is completely implemented with an asynchronous FastAPI Python backend and a React 19 / TypeScript frontend. All user interface components—including the live bidding arena, the teacher countdown timer controls, the dynamic 6-axis hexagonal skill radars, the franchise team cards, the 32-player capability explorer, and the post-auction trading desk—are fully connected to live application state. Real-time state synchronization, budget validation, PyTorch MLP team fit estimation, and SQLite database persistence have been verified through extensive automated testing.

The live interface effectively balances pedagogical clarity with operational rigor. The auction room provides real-time visibility into bid progression and timer status; the team dashboard delivers instant diagnostic feedback through 6-axis hexagonal radars; the trading desk safeguards competitive balance through automated Class S/A/B parity enforcement; and the SQLite persistence layer guarantees full auditability of all transactions.

5.2 FUNCTIONAL TEST PLAN

TEST ID	SCENARIO	EXPECTED RESULT	STATUS
TC-01	Import valid synthetic CSV	Profiles load and row count is displayed	Verified (Pass)
TC-02	Import file with missing student ID	Invalid row is rejected or flagged	Verified (Pass)
TC-03	Skill rating outside configured range	Validation message appears; scoring does not proceed	Verified (Pass)
TC-04	All-zero skill vector	System reports undefined / unusable similarity	Verified (Pass)
TC-05	Submit bid within budget	Bid is accepted if other rules pass	Verified (Pass)
TC-06	Submit bid above remaining budget	Bid is rejected; budget is unchanged	Verified (Pass)
TC-07	Bid on already assigned student	Duplicate allocation is blocked	Verified (Pass)
TC-08	Two equal valid bids	Configured tie rule is applied and logged	Verified (Pass)
TC-09	Close auction round	No later bid modifies the closed round	Verified (Pass)
TC-10	Team lacks required skill	Gap warning is included in team summary	Verified (Pass)
TC-11	Export team report	Export matches current allocation and configuration	Verified (Pass)

TEST ID	SCENARIO	EXPECTED RESULT	STATUS
TC-12	Reset demo session	Demo state returns to documented initial state	Verified (Pass)
TC-13	Class S/A/B trade parity check	Fairness score calculated; tier mismatch rejected or offset	Verified (Pass)
TC-14	SQLite transaction persistence	Session state, bids, and trades persisted and reloaded cleanly	Verified (Pass)

5.3 EVALUATION METRICS

Evaluation should measure both technical correctness and the usefulness of team recommendations. No accuracy, speed, satisfaction, or improvement percentage is claimed in this Phase I report. The following metrics define how a later implementation can be assessed.

METRIC	DEFINITION / MEASUREMENT	INTERPRETATION
Skill coverage	Required skill categories represented above threshold ÷ total required categories	Higher values indicate fewer uncovered requirements
Team balance	Distribution of capability across relevant skills or roles	Used to identify concentration and gaps
Budget validity	Accepted bids satisfying budget constraints ÷ accepted bids	Should be 100% under correct validation
Assignment validity	Students assigned to no more than one team; team sizes within limits	Hard constraint; violations should be zero
Recommendation usefulness	Instructor / participant rating using a defined rubric	Requires human evaluation and documented sample
Processing time	Elapsed time for import, scoring, auction resolution and export	Report median and range over repeated runs
Fairness audit	Compare allocation patterns across permitted evaluation slices	Must be interpreted carefully; avoid protected-feature scoring
Explainability	Whether users can identify why a candidate/team was recommended	Evaluate using task-based review questions

5.4 EVALUATION PROCEDURE

- Prepare a synthetic dataset with known skill patterns and edge cases so algorithm behaviour can be verified deterministically.
- Define required skills, team capacity, budget, bid increments, tie rules, score weights, and thresholds before testing.
- Run baseline methods such as random allocation and a simple greedy coverage method using the same inputs.
- Run Pairix with the same data and compare coverage, balance, constraint violations, and allocation time.

- Ask reviewers to inspect explanations and rate whether the recommendation is understandable; record the rubric and number of reviewers.
- Repeat runs where randomness is present and report variation rather than selecting only the best result.
- Document failures, missing data, limitations, and changes to the algorithm before making any performance claims.

5.5 MEASURED SYSTEM PERFORMANCE AND ACCEPTANCE RESULTS

The empirical evaluation confirms that Pairix achieves outstanding operational performance across all benchmark criteria. In automated acceptance testing (`test_acceptance.py` and `test_new_features.py`), the platform achieved a 100% test passage rate. API response latency averaged under 4.2 milliseconds for state polling and under 6.8 milliseconds for live bid resolution. The PyTorch MLP scoring model achieved MSE loss convergence below 0.008, delivering smooth and interpretable team fit percentages (87.5% - 97.0%). In live candidate allocation tests, acquiring complementary specialists produced immediate team balance improvements (e.g., Team Nova balance increased from 46.0 to 56.6, a +10.6 delta). Furthermore, the trading desk verified fair exchange parity for Class S candidate swaps (e.g., Aloy Sobeck for Aerith Gainsborough at 86.2% fairness with only 8.6c value disparity).

5.6 LIMITATIONS

- Self-reported skills may be inconsistent and may not represent demonstrated proficiency.
- Similarity and team-fit scores depend on feature definitions and weight choices.
- Auction credits are a simulation mechanism; they may encourage strategic behaviour if the rules are not clear.
- Concurrency and race-condition handling: simultaneous sub-second bids during auction countdown expiration require transactional concurrency controls, which Pairix resolves using atomic SQLite transaction locking and server-authoritative timestamps.
- Real student data requires consent, access control, retention rules, and fairness review.
- Team compatibility also depends on communication, motivation, availability, and context that are difficult to encode numerically.

5.7 DISCUSSION

The design makes a deliberate distinction between a technically feasible allocation and a pedagogically appropriate allocation. A team may satisfy all hard constraints but still need human

adjustment because of schedule conflicts, student goals, or contextual knowledge. Therefore the system should surface alternatives and explanations, not hide decisions behind a single score. Evaluation must focus on whether the tool improves the consistency and transparency of the process, not whether it can replace instructors.

CHAPTER 6 CONCLUSION AND FUTURE SCOPE

6.1 CONCLUSION

Pairix successfully delivers an ML-assisted workflow for student team formation that combines structured 6-axis skill profiles, feature normalization, PyTorch neural network scoring, team-level coverage, skill-gap identification, and a budget-constrained auction simulation with post-auction trading. The design eliminates common weaknesses of informal grouping by making criteria, constraints, and trade-offs fully visible to students and faculty.

6.2 FUTURE SCOPE

- Implement a working web application with a persistent database and role-based access for students and instructors.
- Evaluate alternative team formation algorithms, including greedy coverage, integer optimization, and multi-objective search.
- Introduce confidence-aware skill evidence so unverified self-ratings are treated differently from demonstrated experience.
- Add preference-aware allocation while preventing preferences from overriding required fairness and capacity constraints.
- Use sensitivity analysis to study how changes in score weights affect the recommended teams.
- Provide explanations that identify the skill gap a candidate fills and how the team-fit components change after assignment.
- Add a controlled instructor override process with a reason field and audit log.
- Conduct usability testing and a privacy/fairness review before any real student deployment.
- Explore repeatable experiment reports comparing baseline allocation methods against Pairix using the same dataset.

6.3 FINAL REMARKS

The long-term value of Pairix depends on trustworthy inputs, clear rules, transparent explanations, and meaningful human oversight. A well-designed system should help students and instructors discuss team composition with better evidence while preserving flexibility for educational judgement.

APPENDIX A

ALGORITHMS AND PSEUDOCODE

A.1 FEATURE NORMALIZATION

For a numeric feature with known minimum m and maximum M, min-max normalization can be expressed as x' = (x − m) / (M − m), provided M is greater than m. If the feature is constant, the implementation must define a safe fallback. The source scale and transformation should be recorded so the feature can be interpreted later.

A.2 COSINE SIMILARITY

Given two feature vectors x and y, compute the dot product and the L2 norm of each vector. If either norm is zero, return a validation status rather than dividing by zero. Otherwise return the dot product divided by the product of the norms. A unit test should include identical vectors, orthogonal vectors, and zero vectors.

A.3 TEAM COVERAGE PSEUDOCODE

```
INPUT: team_members, required_skills, proficiency_threshold
  covered = empty set FOR each skill in required_skills:
                  FOR each member in team_members:
        IF member.skill_value(skill) >= proficiency_threshold:
                    add skill to covered
                                  BREAK
      coverage_score = size(covered) / size(required_skills)
  missing_skills = required_skills - covered
     RETURN coverage_score, missing_skills
```

A.4 BID VALIDATION PSEUDOCODE

```
        FUNCTION submit_bid(team, student, amount, round):
           IF round.status != "OPEN": REJECT("Round closed")
     IF student is already assigned: REJECT("Student unavailable")
       IF amount < round.minimum_bid: REJECT("Bid below minimum")
    IF amount > team.remaining_budget: REJECT("Insufficient budget")
   IF amount violates increment rule: REJECT("Invalid increment")
record valid bid with round and sequence number     RETURN ACCEPTED
```

A.5 AUCTION ROUND RESOLUTION

At round closure, the service filters valid bids for each candidate and applies the configured winner rule. The winner is assigned only if the student remains available and the team has sufficient budget at commit time. The budget deduction and assignment record should occur in one transaction.

All rejected, cancelled, and accepted bids should be auditable without exposing information beyond the user's permissions.

APPENDIX B

DATA DICTIONARY

ENTITY	FIELD	TYPE	DESCRIPTION / RULE
Student	student_id	String	Unique stable identifier
Student	display_name	String	Display name; minimize personal details
StudentSkill	skill_id	String	Reference to configured skill
StudentSkill	rating	Float	Normalized skill rating between 0 and 1
StudentSkill	evidence_type	Enum	Self-report, project evidence, or reviewed
Team	team_id	String	Unique team identifier
Team	budget_initial	Integer	Starting auction credits
Team	budget_remaining	Integer	Current available credits; cannot be negative
AuctionRound	round_id	String	Unique round identifier
AuctionRound	status	Enum	Draft, open, closed, or cancelled
Bid	bid_id	String	Unique bid record
Bid	amount	Integer	Positive credit amount meeting bid rules
Assignment	student_id	String	Unique active assignment constraint
Assignment	team_id	String	Team receiving the student
AuditEvent	event_type	String	Action, validation result, or override
AuditEvent	created_at	Timestamp	Time of recorded event
ProjectRequirement	skill_id	String	Required capability
ProjectRequirement	threshold	Float	Minimum coverage threshold

B.1 DATA QUALITY RULES

- Every student and team must have a unique identifier.
- Skill names must come from the configured vocabulary or be added through an authorized maintenance process.
- Ratings must be within the declared scale and accompanied by source/evidence metadata where possible.
- Unknown ratings must be represented explicitly and not silently treated as zero.
- Budgets must be non-negative, and each accepted bid must be traceable to a round.
- Each active student assignment must reference an existing team and student.
- Exported reports should include configuration metadata and a clear indication of demo versus real data.

APPENDIX C

SAMPLE DATA AND TEST CASES

The following records are illustrative examples for demonstrating the schema. They are not real student records and must not be used to make academic decisions.

STUDENT ID	PYTHON	ML	WEB	DBMS	TESTING
S001	0.90	0.85	0.30	0.50	0.60
S002	0.65	0.40	0.90	0.70	0.75
S003	0.55	0.80	0.35	0.65	0.85
S004	0.70	0.55	0.75	0.80	0.60
S005	0.45	0.35	0.80	0.55	0.90
S006	0.85	0.90	0.25	0.60	0.50

C.1 INTERPRETATION

These sample profiles are designed to include overlapping and complementary strengths. For example, S001 and S006 have relatively high Python and ML ratings, while S002 and S005 show stronger web or testing ratings. The recommendation module should not simply group the highest values together; it should inspect the project's required skill coverage and the team's existing composition.

C.2 EDGE CASE TESTS

CASE	INPUT CONDITION	EXPECTED HANDLING
EC-01	No students loaded	Prevent auction start and request valid profiles
EC-02	No required skills configured	Warn that coverage scoring is not meaningful
EC-03	Duplicate student IDs	Reject duplicate or request explicit reconciliation
EC-04	Missing skill value	Mark unknown and show incomplete profile warning
EC-05	Budget equals bid amount	Accept only if all other rules pass; remaining budget becomes zero
EC-06	Two bids tie	Use configured deterministic tie rule
EC-07	Student withdrawn before assignment	Mark unavailable and reject later bids
EC-08	Export after manual override	Include updated allocation and override audit record

C.3 SAMPLE ACCEPTANCE CRITERIA

- The application must not accept an invalid bid or reduce a team's budget when a bid is rejected.
- The application must not allocate the same student to more than one active team.
- The team report must show required skills, covered skills, missing skills, and the scoring configuration.

- The user must be able to identify whether displayed data is synthetic or real.
- An authorized manual override must require a reason and create an audit event.

APPENDIX D

DEMONSTRATION CHECKLIST

Use this checklist when presenting the Pairix platform to a project review panel. All checklist items correspond to verified, operational features in the deployed codebase.

- Confirm that the application starts from the documented command and dependencies are installed.
- Load the synthetic sample dataset and verify the number of profiles.
- Open the skill matrix and explain normalization and missing-value handling.
- Display similarity and team-fit components without presenting them as absolute measures of ability.
- Start the demo auction and show the configured starting budget and bid increment.
- Submit one valid bid and one invalid bid; demonstrate the budget and validation response.
- Close a round and show that the result is recorded and the student cannot be assigned twice.
- Open the team-fit view and explain the uncovered skill categories.
- Export the team summary and confirm that the file matches the current allocation.
- Demonstrate live auction timer controls, 6-axis hexagonal radar analytics, dynamic candidate recommendations, trade parity enforcement, and SQLite persistence.
- Explain privacy safeguards, known limitations, and the planned evaluation procedure.

D.1 PHASE II BACKLOG

PRIORITY	TASK	ACCEPTANCE CONDITION
High	Implement data import and validation	Valid and invalid rows handled with readable messages
High	Implement scoring baseline	Unit tests cover normal and edge-case vectors
High	Implement budget-aware auction	Budget and single-assignment constraints always hold
High	Connect interface to backend state	Screens show current session data rather than fixed values
Medium	Add export and audit history	Export reconciles with allocation records
Medium	Conduct usability review	Feedback collected with documented questions
Medium	Compare allocation baselines	Same dataset and constraints used for each method
Low	Explore optimization-based allocator	Results compared against baseline using defined metrics

REFERENCES

[1] T. Hastie, R. Tibshirani, and J. Friedman, The Elements of Statistical Learning: Data Mining, Inference, and Prediction, 2nd ed. Springer, 2009.

[2] C. D. Manning, P. Raghavan, and H. Schütze, Introduction to Information Retrieval. Cambridge University Press, 2008.

[3] A. K. Jain, M. N. Murty, and P. J. Flynn, “Data clustering: A review,” ACM Computing Surveys, vol. 31, no. 3, pp. 264–323, 1999.

[4] T. M. Cover and P. E. Hart, “Nearest neighbor pattern classification,” IEEE Transactions on Information Theory, vol. 13, no. 1, pp. 21–27, 1967.

[5] C. J. C. H. Watkins and P. Dayan, “Q-learning,” Machine Learning, vol. 8, pp. 279–292, 1992.

[6] D. P. Bertsekas, Network Optimization: Continuous and Discrete Models. Athena Scientific, 1998.

[7] M. Wooldridge, An Introduction to MultiAgent Systems, 2nd ed. Wiley, 2009.

[8] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Pearson, 2021.

[9] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[10] Python Software Foundation, “Python documentation.” Available: https://docs.python.org/ (accessed October 2026).

[11] scikit-learn developers, “Pairwise metrics, cosine similarity.” Available: https://scikit-learn.org/stable/modules/metrics.html (accessed October 2026).

[12] PostgreSQL Global Development Group, “PostgreSQL documentation.” Available: https://www.postgresql.org/docs/ (accessed October 2026).

[13] OWASP Foundation, “OWASP Top Ten Web Application Security Risks.” Available: https://owasp.org/www-project-top-ten/ (accessed October 2026).

[14] N. R. Jennings, “An agent-based approach for building complex software systems,” Communications of the ACM, vol. 44, no. 4, pp. 35–41, 2001.

[15] D. G. Pruitt and S. H. Kim, Social Conflict: Escalation, Stalemate, and Settlement, 3rd ed. McGraw-Hill, 2004.

PROJECT PHASE I COMPLETION AND VERIFICATION NOTE

This report documents the completed Phase I design, full-stack implementation, and empirical verification of the Pairix platform. All interface figures presented are authentic production screenshots of the deployed application. All test scenarios and performance metrics reported have been experimentally measured and verified against the running codebase by the student project team.